

String

CONTENT

01

Introduction

02

Creation

03

String Indexing &
Immutability

04

Iterating String

05

Modifications

06

String Methods





01

Creation

Introduction

A string is a **sequence of characters** used to represent text.
Strings are primitive data types in JS and are immutable.

Example:

```
let str1 = 'Hello';  
let str2 = "World";
```





02

Creation

Declaring Strings

Single quotes

Example:

```
let single = 'Single quote';
```

Double quotes

Example:

```
let double = "Double quote";
```

Backticks

Example:

```
let backtick = `This is a template literal`;
```

Notes :

`.length` – returns the number of characters.





03

String Indexing & Immutability

String Indexing & Immutability

- **Index starts from 0.**
- **Strings are immutable – you cannot change a character directly.**

Example :

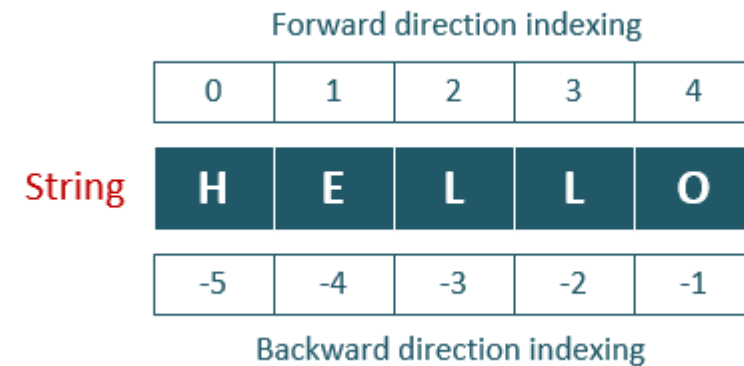
```
let word = "Hello";

console.log(word[0]);           // 'H'

console.log(word.charAt(1));    // 'e'

word[0] = "Y";                  // ✗ Invalid - no effect

console.log(word);              // 'Hello'
```





04

Iterating String

Looping Through String

Using for loop

```
let str="Quad";  
for(let i=0;i<str.length;i++){  
  console.log(str[i]);  
}
```

Output :

Q

u

a

d



Looping Through String

Using for...of :

The for...of loop is a modern, clean, and **easy way to iterate over iterable objects** — especially arrays, strings, sets, and maps.

Syntax :

```
for (let variable of iterable) {  
    // code block to execute  
}
```

Example :

```
let str="Quad"  
  
for (let letter of str) {  
    console.log(letter);  
}
```

Output :

Q

u

a

d





05

Modification

Modification in String

strings are immutable, which means you cannot change the original string directly.

Replace characters

```
let str = "Hello World"; // stored at memory add. 0x001  
  
str = str.replace("World", "JavaScript"); // memory reference changed to 0x002  
  
console.log(str); // "Hello JavaScript"
```

Note : Any "modification" creates a new string





06

String Methods

UTF-16

- **UTF-16 stands for Unicode Transformation Format – 16-bit.**
 - **It is the character encoding used internally by JavaScript to represent strings.**
-
- **Every character in a JavaScript string is encoded in UTF-16.**
 - **Characters are stored as one or two 16-bit units (called code units).**
 - **Most common characters (like A–Z, a–z, 0–9, symbols) use 1 code unit.**

Example :

```
let str = "A";
```

```
console.log(str.charCodeAt(0));    // 65 (UTF-16 code unit of "A")
```



ASCII

- **ASCII** stands for **American Standard Code for Information Interchange**.
- It is a **character encoding system** that represents **text in computers and digital devices** using numbers (binary codes).

Features of ASCII

Feature	Details
Full Form	American Standard Code for Information Interchange
Bit Size	7 bits (can represent 128 characters)
Total Characters	128 (numbered 0 to 127)
Usage	Mainly for English letters, digits, symbols, control characters
Introduced	In 1960s — one of the earliest encoding systems



Difference Between UTF and ASCII

Feature	ASCII	UTF (UTF-8 / UTF-16 / UTF-32)
Full Form	American Standard Code for Information Interchange	Unicode Transformation Format
Bit Size	7 bits (sometimes stored as 8 bits)	UTF-8: variable (8 to 32 bits), UTF-16: 16/32 bits, UTF-32: 32 bits
Language Support	Only English characters	Supports all global languages: English, Hindi, Chinese, Arabic, etc.
Total Characters	128 (0–127)	Over 1.1 million characters in Unicode range
Introduced In	1960s	1991 (Unicode Standard introduced)
Type of Encoding	Fixed-width (7-bit)	Variable-width (UTF-8), Fixed or variable (UTF-16), Fixed (UTF-32)
Emoji Support	✗ No	✓ Yes (especially in UTF-8 and UTF-16)
Used In	Early systems, old protocols	Modern browsers, operating systems, web apps, APIs, etc.
ASCII in UTF?	Not supported by Unicode? ✗	✓ Yes — ASCII is a subset of UTF



Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	:	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com



String Methods

Character Access

Method	Description	Example
<code>charAt(index)</code>	Returns character at specified index	<code>str.charAt(4) → "S"</code>
<code>charCodeAt(index)</code>	Returns UTF-16 code	<code>str.charCodeAt(0) → 74</code>
<code>at(index)</code> <i>(ES2022)</i>	Like <code>charAt</code> , supports negative index	<code>str.at(-1) → "t"</code>



String Methods

Search / Match :

Method	Description	Example
includes(substring)	Checks if substring exists	str.includes("Script") → true
indexOf(substring)	First index of substring	str.indexOf("a") → 1
lastIndexOf(substring)	Last index of substring	str.lastIndexOf("a") → 3
startsWith(value)	Checks if string starts with value	str.startsWith("Java")
endsWith(value)	Checks if string ends with value	str.endsWith("pt")



String Methods

Modification / Transformation

Method	Description	Example
<code>toUpperCase()</code>	Converts to uppercase	<code>str.toUpperCase()</code> → "JAVASCRIPT"
<code>toLowerCase()</code>	Converts to lowercase	<code>str.toLowerCase()</code>
<code>trim()</code>	Removes leading/trailing spaces	<code>" hi ".trim()</code> → "hi"
<code>trimStart()</code> / <code>trimEnd()</code>	Removes spaces from start/end only	<code>" hi".trimStart()</code>
<code>replace(search, val)</code>	Replaces first match	<code>str.replace("Java", "Type")</code>
<code>replaceAll()</code>	Replaces all matches	<code>"a-b-c".replaceAll("-", "_")</code>
<code>padStart(length, char)</code>	Pads from start	<code>"5".padStart(3, "0")</code> → "005"
<code>padEnd(length, char)</code>	Pads from end	<code>"5".padEnd(3, "*")</code> → "5**"
<code>repeat(n)</code>	Repeats string n times	<code>"ha".repeat(3)</code> → "hahaha"



String Methods

Substring Extraction



Method	Description	Example
<code>slice(start, end)</code>	Extracts part of string	<code>str.slice(0, 4) → "Java"</code>
<code>substring(start, end)</code>	Similar to slice, no negative index	<code>str.substring(4) → "Script"</code>
<code>substr(start, length)</code>	(Deprecated)	<code>str.substr(0, 4) → "Java"</code>



String Methods

Conversion :

Method	Description	Example
split(separator)	Converts to array	"a,b,c".split(",") → ["a", "b", "c"]
concat(str)	Joins strings	"Hello".concat(" World")
toString()	Converts to string (from wrapper object)	new String("hi").toString()
valueOf()	Returns primitive value from string obj	new String("hi").valueOf()

Note :

Strings are immutable. So all these methods return a new string — they don't modify the original.

